How Far from Easy?
Structural Distance in Subset Sum

Jonathan R. Landers

October 2026

Companion note V · Working draft

Abstract. How far can a subset-sum list depart from an easily solved structure before the remaining search becomes large? We make this question concrete by finding a largest superincreasing core and searching its exceptions. Sorted greedy discovery takes $O(n \log(n+1))$ operations. Logarithmically many exceptions give a sufficient polynomial regime; merging equal exceptional sums can extend it. A frozen comparison then tests whether choosing a different core helps. Two selectors reduce residual support on 18 of 54 structured lists, but neither is faster on any of their 162 target cases. Their paired median total-time ratios to the simple core solver are about nine. The constructive guarantee holds; these attempts to improve the decomposition do not repay their cost.

A list can be mostly easy

How far can a list depart from an easily solved structure before the unresolved search becomes large?

In subset sum, a *weight* is simply one of the numbers in a list. Given positive integer weights $w_1, \ldots, w_n$ and an integer target t, we may take each listed item once or leave it out. We ask whether some selection adds up to t. Equal numbers are still separate copies. The target and list are fixed throughout this note.

There are 2^n selections. The first note's coverage framing asks how many can be handled without checking them individually, while charging for the work that makes this possible. The experiments in notes II–IV show why that last condition matters. Here we choose a structure whose solution is easy to explain, whose presence can be checked, and whose distance from an arbitrary list can be measured.

Consider the list

$$[2,\ 5,\ 6,\ 11,\ 23].$$

Set aside 6. Every number in the remaining list $[2, 5, 11, 23]$ exceeds the sum of all smaller numbers in that list. This property is called *superincreasing*. It makes the choices forced when we work downward from the largest number.

If the largest number exceeds the target remainder, we must leave it out. Otherwise we must take it: all smaller numbers together cannot reach even that largest number. Subtract it and continue. A zero remainder gives a solution; a nonzero remainder at the end proves that none exists.

For target 17, the easy part alone fails. We therefore try the other possibility for the exceptional item: take 6, leaving target 11. The easy part reaches 11, giving the original solution $6 + 11 = 17$. We have replaced the full search by two simple questions.

Finding the easy part is also easy

A sorted core $g_1, \ldots, g_m$ is superincreasing when

$$g_j > \sum_{h<j} g_h.$$

We can find a largest such core by sorting the full list, starting with total zero, and keeping a number exactly when it exceeds the total already kept. Put every rejected item among the exceptions.

Proposition 1. This scan finds a maximum-size superincreasing core. Including sorting, discovery takes $O(n \log(n+1))$ arithmetic/comparison operations.

Why it works. Compare the greedy choices with any other valid core, in increasing order. Greedy's first choice is no larger. Suppose its first $j-1$ choices have appeared no later and have no greater total. The other core's jth number exceeds its own preceding total, so it also exceeds greedy's. Greedy can therefore accept a jth number no later, and no larger. Induction proves that greedy reaches every feasible length. Its own choices satisfy the defining inequalities, so its length is maximum. The scan is linear after sorting. □

This is stronger than merely recognizing a supplied easy list. The algorithm discovers the core itself. We independently verify its inequalities and its partition of the indexed input. Exhaustive checks on small lists also confirm the maximum-cardinality claim. The empty core and target zero are handled as ordinary boundary cases.

Count what remains

Let C be the discovered core and E its complement. Define

$$k(w) = \text{fewest items removable to leave a superincreasing list.}$$

Proposition 1 gives $|E| = k(w)$. For each selection of exceptional items with sum s, ask whether the core reaches $t - s$. The original answer is YES exactly when one of these core queries succeeds.

There are at most $2^{k(w)}$ exceptional selections. Each core query takes $O(n+1)$ operations. Thus sorting, core discovery, and residual search have the operation bound

$$O\Big(n \log(n+1) + (n+1)2^{k(w)}\Big).$$

The count describes structural search. Including encoded-integer arithmetic, input checks, and root gcd checks gives a conservative bit-cost bound $\text{poly}(L)2^{k(w)}$, where L is the full binary input length.

A sufficient polynomial regime. If a family has $k(w) = O(\log(n+1))$, the exact method, including discovery of its easy core, takes polynomial time in the input length.

This is a guarantee for the family, including NO answers. A large value of k does not prove that another algorithm must struggle.

Exception count is only a first measure. Distinct exceptional selections can have the same sum, and hence ask exactly the same question of the core. Write

$$\mathcal{A}(E) = \left\{ \sum_{i \in B} w_i : B \subseteq E \right\}, \qquad M(E) = |\mathcal{A}(E)|.$$

If twelve exceptional items all equal the same positive value, their sums are just zero through twelve copies of that value: $M = 13$, rather than $2^{12} = 4096$ possibilities.

Construct the distinct sums from $\{0\}$, merging the sorted list with a copy shifted by each next weight. Retain one indexed witness per sum. Every prefix sum survives in the final set, bounding intermediate lists by $M(E)$ and construction by $O(|E|M(E))$. Querying once per distinct sum gives

$$O(n \log(n+1) + (n+1)M(E))$$

for the structural algorithm, or conservatively $\text{poly}(L)M(E)$ in the bit model. A polynomially bounded residual support is sufficient even with many exceptions.

A smaller remainder may be expensive to find

A largest core need not leave the fewest distinct sums. For

$$[1, 1, 1, 2, 3],$$

greedy keeps $[1, 2]$; its exceptions $[1, 1, 3]$ have six sums, $0, \ldots, 5$. The equally large core $[2, 3]$ leaves $[1, 1, 1]$, with only four sums. Neither core can be enlarged to three items.

This suggests searching for a better decomposition. We test two explicit policies. Both scan single-item exchanges between the greedy core and its exceptions, verify each candidate, and preserve the maximum core size. The first chooses by a cheap bound obtained from value multiplicities: count how many copies of each value could be taken. The second constructs the actual supports of at most four candidates and selects the smallest, reusing its representatives.

The second policy must pay for every support it probes, including losing candidates. With candidate support sizes $M_1, \ldots, M_b$, its construction work depends on $\sum_j M_j$, not only on $\min_j M_j$. A better final representation does not make its discovery free.

A comparison that charges for structure

We froze the protocol, generators, solver variants, reporting plan, and source hashes before generating 60 fresh lists at $n = 20, 28, 36$. Fifty-four lists combine a superincreasing core with $4, 8$, or 12 injected items. The families use unrelated exceptions, repeated exceptions, and the exchange example above extended with larger superincreasing weights. Six random 41-bit lists provide unstructured controls. Actual deletion distance is measured; it need not equal the injected count.

Each list supplies three targets: a random-subset target, a uniform integer from the middle 40% of its total, and a target from that same range accepted only if it passes reachable-residue checks modulo $8, 10, 16, 32$ and an independent exact oracle labels it NO. Lists have no common divisor greater than one. This deliberately conditioned NO cohort tests more than immediate root exclusions; it does not estimate unconditional average performance.

The 180 targets comprise 60 YES and 120 NO cases. The unconditioned uniform targets happened to be NO throughout this sparse synthetic sample. Every planned conditioned-NO slot was filled, and no new list duplicates a pilot list.

Seven methods include the two simple core solvers, the two selectors, memoized bounded depth-first search (DFS), meet-in-the-middle (MITM), and integer-bitset dynamic programming where its width cap permits it. Each timing uses fresh preprocessing and includes discovery, verification, selection, construction, and queries. Three shuffled-order repetitions per case give 3,780 runs. Comparisons use per-case medians and require both methods to complete every repetition.

All 3,117 completed timing answers agree with independent binary-mask half-enumeration labels, and every returned witness is valid. The other 663 runs reached declared work, time, or bitset limits; they return unresolved. Sixteen independent test groups passed before the freeze.

What the measurements say

Both selectors reduce support on 18 structured lists, all from the predeclared exchange family. There the residual support falls from $k + 3$ to $k + 1$. Yet neither selector beats the simple greedy-core distinct-sum solver on any of the 162 structured target cases.

Table 1: All completed paired cases. Ratios compare total time with the simple greedy-core distinct-sum solver; smaller than 1 favors selection.

Method	Completed	Wins / paired	Median ratio
Simple greedy core	168/180	—	1.000
Multiplicity-ranked core	168/180	5/168	9.034
Exact-support probes	162/180	0/162	9.211

The cheap selector's five wins are on unstructured 20-item controls with the same selected core and the same $M = 262144$ as the simple method. They do not demonstrate a gain from a better decomposition. Across matched cases, the selectors' median time ratios are about nine. Candidate selection itself takes median times of roughly 1.91 and 2.38 ms, compared with about 0.10 ms for their completed queries. These component medians are descriptive, not a subtraction of matched total times.

Ordinary search remains competitive. The simple distinct-sum core solver completes 168/180 cases and beats the fastest feasible elementary baseline (DFS, MITM, or bitset) on 31 of those paired cases; its median ratio to that baseline is 1.899. MITM completes every case, including the larger-distance controls on which core methods reach limits. We therefore have a constructive special-case guarantee, with no general practical speed advantage established by this comparison.

Memory tells a related story. A separate pass profiles four predeclared NO cases, one per family at $n = 20$, giving 28 profiles. Time limits are disabled during tracing; the other limits remain. All 24 completed answers are correct. On the unstructured case, storing exceptional sums peaks at 34.3 MiB of new Python-traced allocations. Exact support selection reaches 67.5 MiB and still stops at its work limit. Visiting exceptional selections one at a time uses about 6 KiB there, while paying for repeated queries in time. These are allocation diagnostics on four cases, not process-memory or universal rankings.

The lesson for certified coverage

The constructive part answers the opening question for one precise structure. We can discover a largest tractable core cheaply. Logarithmically many remaining binary choices give polynomial work. Many more exceptional items can also be manageable when their distinct sums merge into a small set.

The experiment asks what happens when we try to improve that representation. It finds genuine support reductions, but these particular selectors spend more obtaining them than they save. The costed coverage framing therefore becomes operational: record the unresolved possibilities *and* the full effort needed to expose them.

This note uses established ingredients. Superincreasing membership is classical [1]; branching on exceptional choices has relatives in backdoors to tractable classes [2]; support-sensitive subset-sum algorithms are a substantial existing subject [3,4]. We do not establish novelty of the greedy core lemma, a new general complexity bound, or a universal speedup.

The next useful improvement must earn its discovery cost: cheaper recognition, a structure removing much more work, or an adaptive procedure that exposes it only where needed. The present evidence supports retaining the simple core solver alongside ordinary exact baselines while those directions are investigated.

Reproducibility. Full proofs, the local freeze, saved inputs and labels, every timing repetition, memory profiles, and the claim-by-claim literature audit are in the companion experiment package. This is a working PDF/LaTeX draft; the published website remains the four-note sequence.

References

1. SageMath. *Knapsack problems: Superincreasing sequences.* `https://doc.sagemath.org/html/en/reference/numerical/sage/numerical/knapsack.html`.
2. Ryan Williams, Carla P. Gomes, and Bart Selman. *Backdoors To Typical Case Complexity.* IJCAI, 2003. `https://www.cs.cornell.edu/selman/papers/pdf/03.ijcai.backdoors.pdf`.
3. Karl Bringmann and Vasileios Nakos. *Top-k-Convolution and the Quest for Near-Linear Output-Sensitive Subset Sum.* Full version of STOC 2020. `https://arxiv.org/abs/2107.13206`.
4. Karl Bringmann, Nick Fischer, and Vasileios Nakos. *Beating Bellman's Algorithm for Subset Sum.* SODA 2025. `https://arxiv.org/abs/2410.21942`.